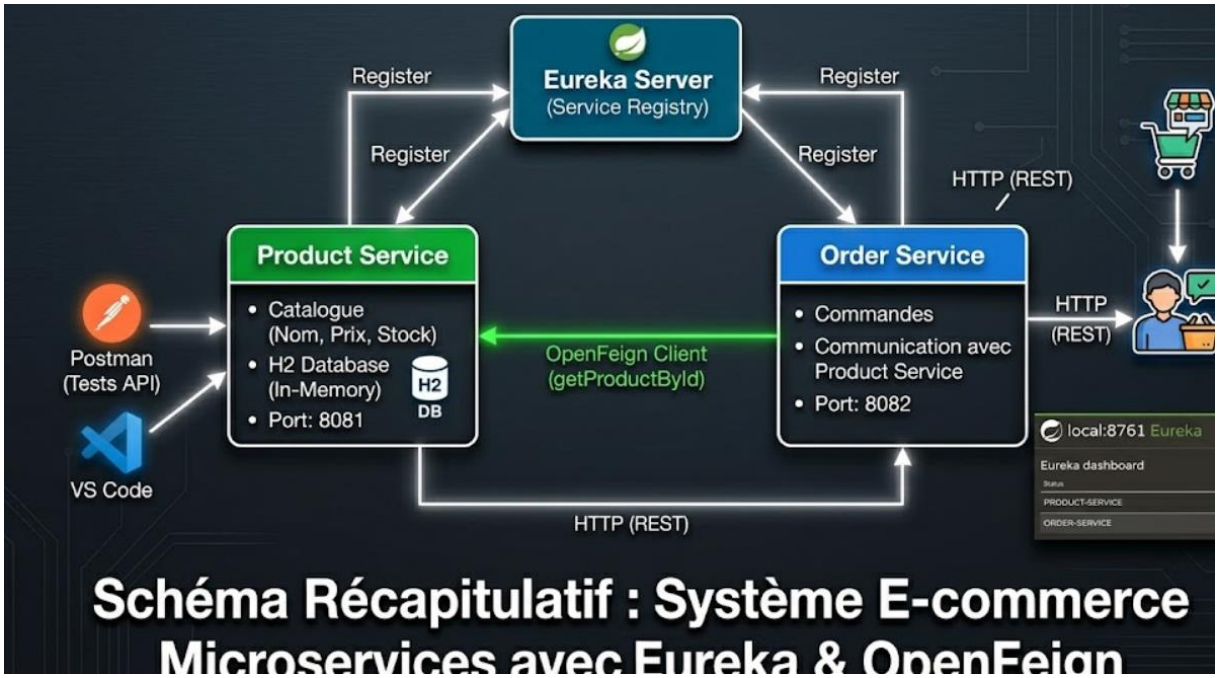
المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵉⵏⵎⵍⴰⵍ - ⴱⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

Filière : Cycle d'ingénieur - IA & Cybersécurité

Rapport de TP4 : Architecture Microservices avec Spring Cloud



Réalisé par : Ibtissam El Fahmi

Encadrée par : Encadrée par : Prof. BE ELBAGHAZAOUI

1. Introduction

Ce rapport présente la mise en place d'une architecture microservices pour une application d'E-commerce simplifiée. L'objectif principal est de comprendre comment diviser une application monolithique en services indépendants capable de communiquer entre eux via un annuaire de services (**Eureka**) et un client HTTP déclaratif (**OpenFeign**).

2. Architecture du Projet

L'écosystème est composé de trois services principaux :

- **Eureka Server :** Le registre de services qui permet la découverte dynamique.
- **Product Service :** Gère le catalogue des produits (Nom, Prix, Stock).
- **Order Service :** Gère la création des commandes et communique avec le service produit pour valider les prix et les informations.

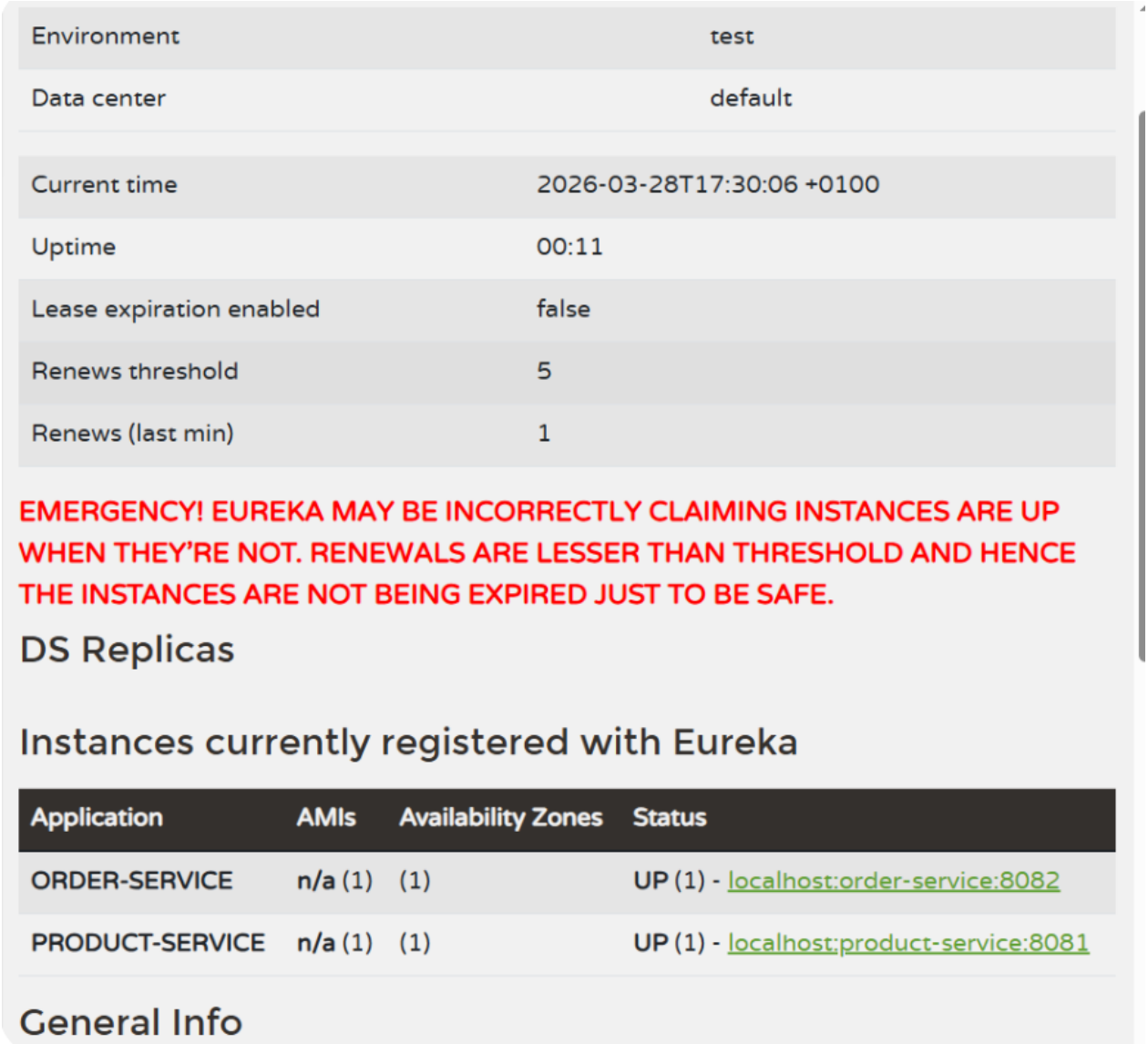
3. Technologies Utilisées

- **Framework :** Spring Boot 3.x
- **Service Discovery :** Spring Cloud Netflix Eureka
- **Communication :** Spring Cloud OpenFeign
- **Base de données :** H2 Database (In-Memory)
- **Outils :** Postman (Tests API), VS Code.

4. Réalisation Technique

A. Configuration d'Eureka Server

Le serveur Eureka a été configuré pour centraliser les instances. Une fois lancé, les services s'y enregistrent automatiquement.



Environment	test
Data center	default

Current time	2026-03-28T17:30:06 +0100
Uptime	00:11
Lease expiration enabled	false
Renews threshold	5
Renews (last min)	1

EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT BEING EXPIRED JUST TO BE SAFE.

DS Replicas

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ORDER-SERVICE	n/a (1)	(1)	UP (1) - localhost:order-service:8082
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - localhost:product-service:8081

General Info

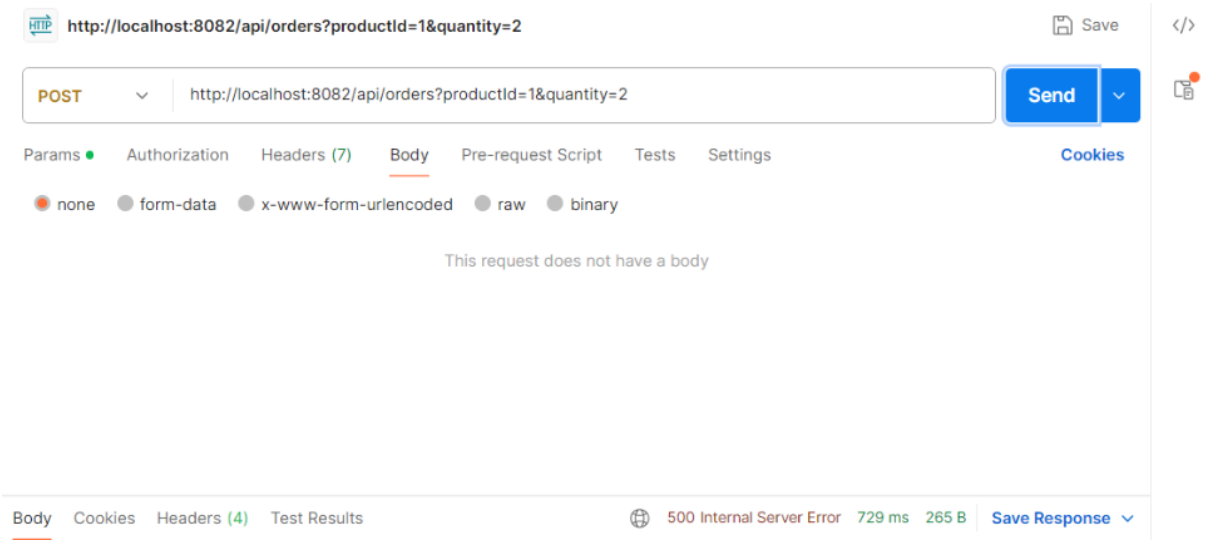
B. Communication Inter-service avec OpenFeign

Pour permettre à `Order Service` de récupérer le prix d'un produit, nous avons défini une interface `ProductClient` :

Java
```
@FeignClient(name = "product-service")
public interface ProductClient {
    @GetMapping("/api/products/{id}")
    ProductDTO getProductById(@PathVariable("id") Long id);
}
```

C. Résolution des problèmes techniques (Challenge)

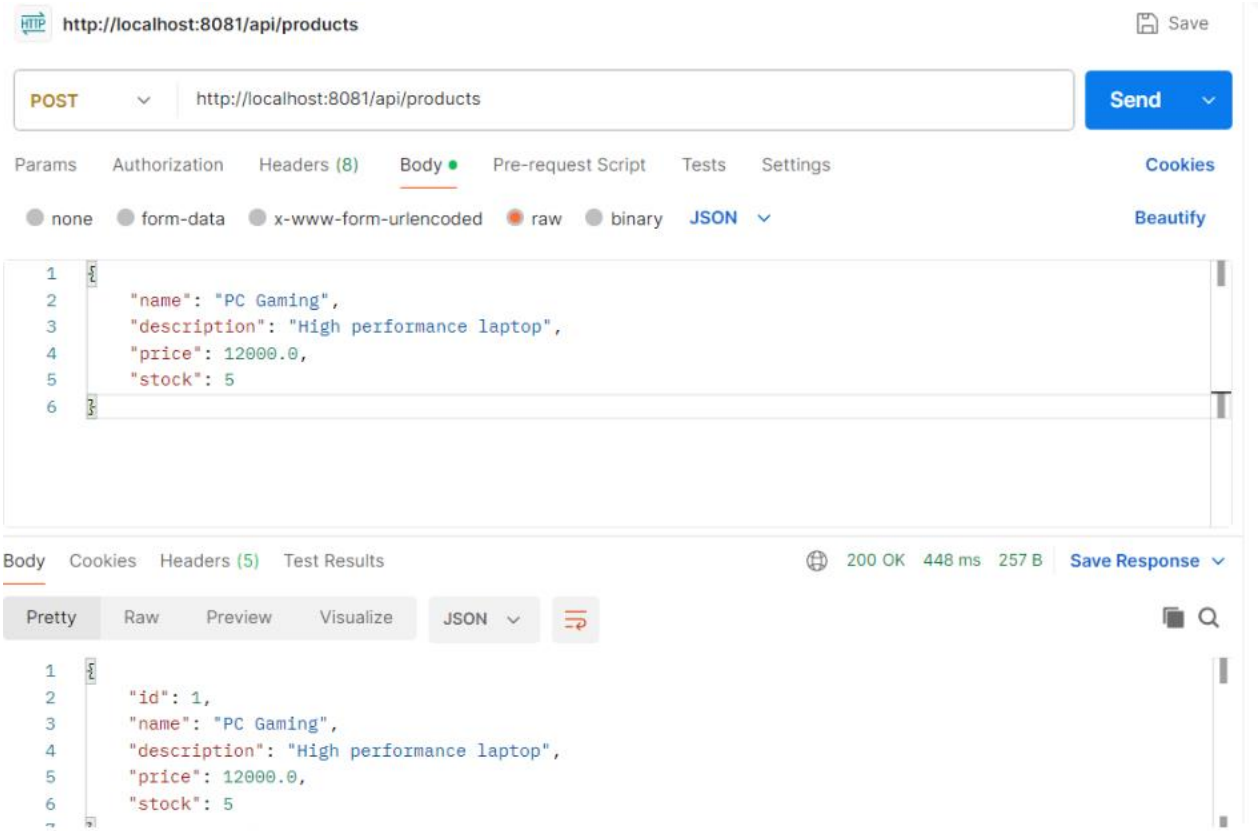
Lors des tests, nous avons rencontré une erreur **500 Internal Server Error**. Après analyse des logs, le problème venait de l'absence d'un point de terminaison (Endpoint) spécifique dans le `ProductController`. La solution a été d'implémenter la méthode `getProductById` pour répondre aux requêtes du client Feign.



5. Tests et Résultats (Démonstration)

Étape 1 : Création d'un produit

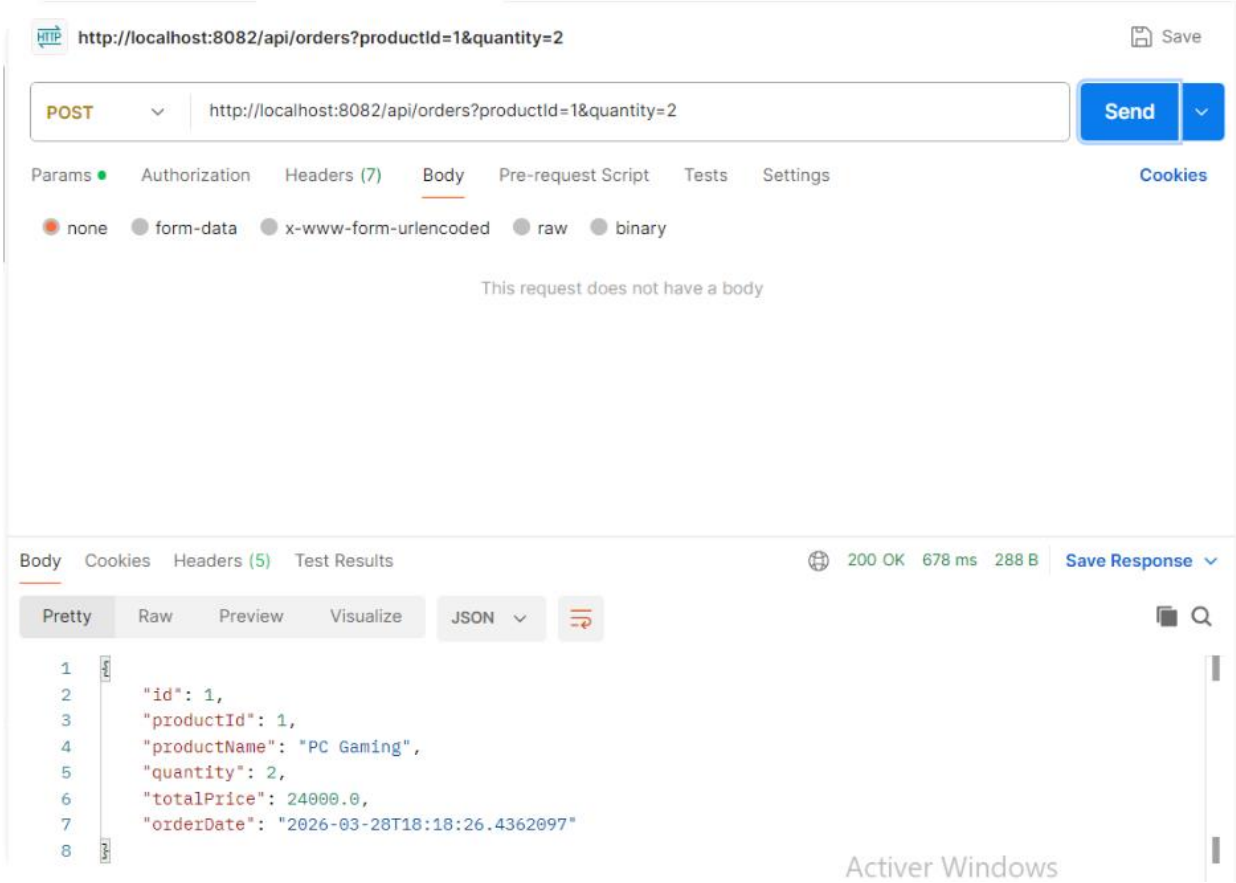
Nous avons ajouté un produit "PC Gaming" via une requête POST.



Étape 2 : Création d'une commande

En appelant le service `Order Service` sur le port 8082, le système a calculé automatiquement le montant total en communiquant avec le service produit.

- **Prix unitaire :** 12000.0
- **Quantité :** 2
- **Total calculé :** 24000.0



6. Conclusion

Ce TP nous a permis de maîtriser les concepts fondamentaux des microservices. Nous avons appris à gérer la découverte de services et la communication synchrone. Cette architecture offre une grande flexibilité et une scalabilité nécessaire pour les applications modernes de grande envergure.